

**Московский Физико-Технический Институт
Физтех-школа Аэрокосмических технологий
Институт Аэромеханики и Летательной Техники**



ФАКТ

**Архитектура Компьютера
и Операционные Системы.
Часть 2. Основы операционных систем**

Лекция 7: Очереди сообщений

**Новиков Андрей Валерьевич
д.ф.-м.н.**

Жуковский

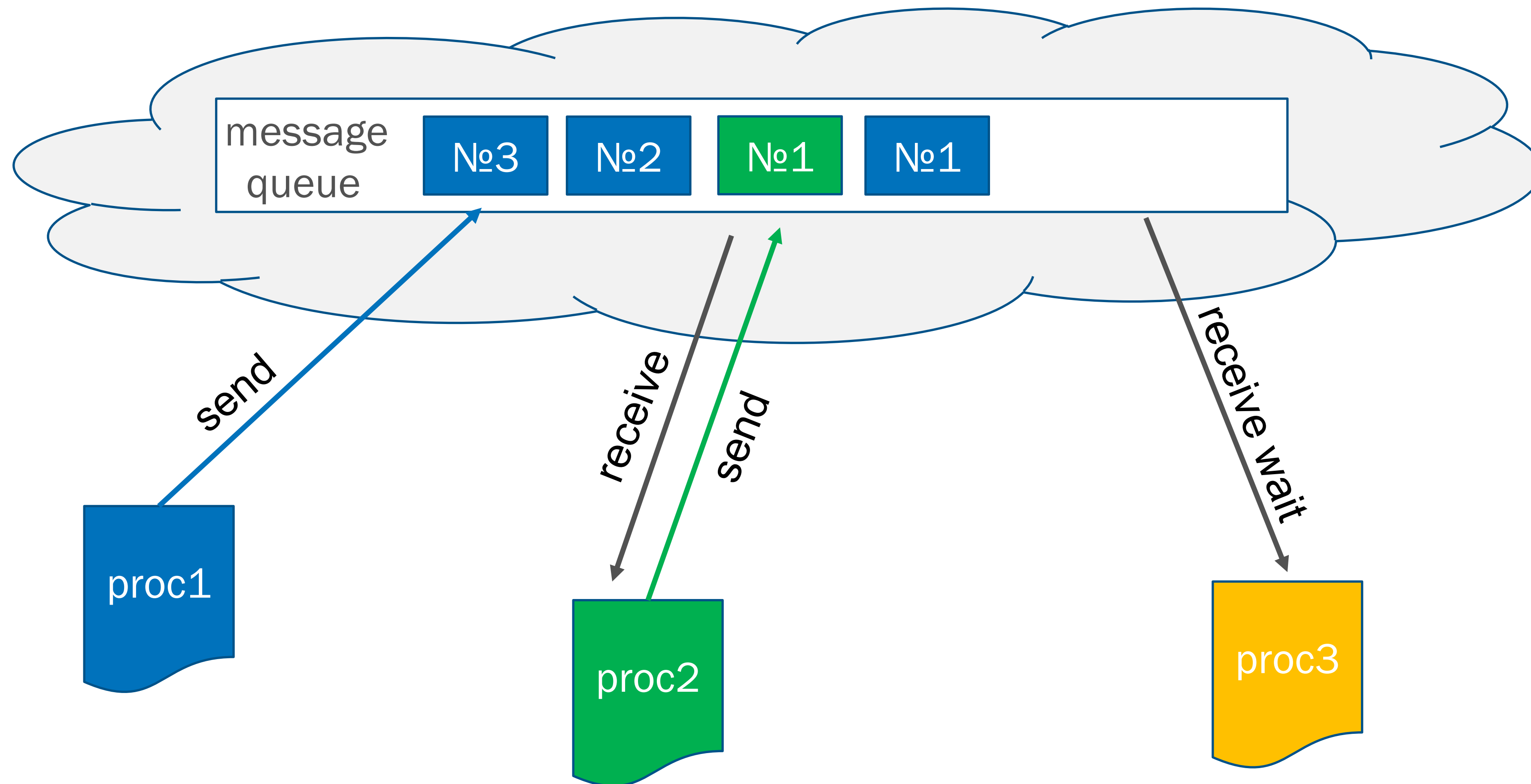
System V IPC

- ❑ Средства связи System V IPC:
 - совместно используемая (разделяемая) память – shared memory;
 - семафоры – semaphores;
 - **очереди сообщений – message queues.**

Модель сообщений для взаимодействия процессов

- ❑ На передаваемую информацию накладывается определённая структура:
 - известно где заканчивается одна порция данных, и начинается другая.
- ❑ Возможно использовать одну линию связи для двунаправленной передачи между несколькими процессами.
- ❑ Возможно использовать для синхронизации процессов.
 - встроенные механизмы блокировки и взаимного исключения.

Модель сообщений для взаимодействия процессов

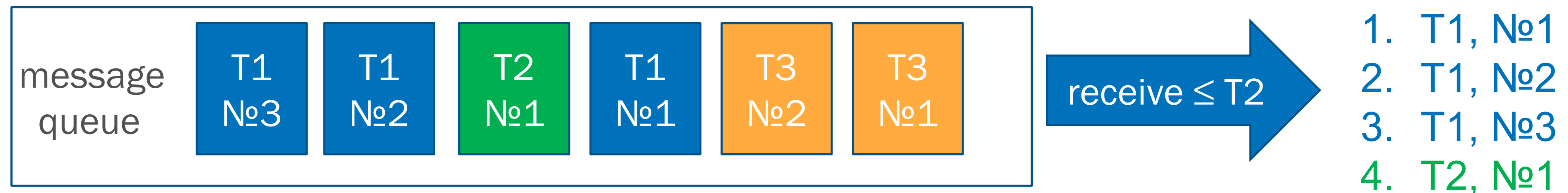


Очереди сообщений System V IPC: message queues

- ❑ Размещаются в адресном пространстве ядра
 - возможно взаимодействие процессов, работающих не одновременно
 - размер ограничен, задаётся администратором
- ❑ Пространство имён – множество значений ключа, генерируемого ftok()
- ❑ Реализованы в виде однонаправленных списков. Элемент списка – сообщение.
- ❑ Сообщения имеют атрибут «тип сообщения», размер не фиксирован.
- ❑ Операции помещения (send) и получения (receive) сообщения в/из очереди
 - атомарные, взаимоисключающие
 - блокирующие при пустой или переполненной очереди.

Очереди сообщений System V IPC: message queues

- Способы выборки сообщения из очереди (receive)
 - В порядке FIFO независимо от типа сообщения
 - В порядке FIFO для сообщений конкретного типа
 - В порядке FIFO для сообщений с типом не превышающим заданный



Консольная команда для очередей сообщений SysV IPC

```
$ ipcs -t|-p|-c|-l|-u
```

Информация об объектах SysV IPC

- ❑ **-t** – показать время последней операции
- ❑ **-p** – PID процесса создателя и совершившего последнюю операцию
- ❑ **-l** – системные лимиты для объектов SysV IPC, в частности **размер очереди сообщений**
- ❑ **-u** – суммарная информация (summary)

Получение доступа / создание очереди сообщений SysV IPC

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>

int msgget(
    key_t key,
    int msgflg
);
```

Получает доступ или создаёт очередь сообщений

- ❑ Возвращаемое значение
 - >0 – дескриптор SysV IPC
 - 1 – ошибка, код ошибки в `errno`
- ❑ `key` – уникальное имя (ключ), созданное `ftok()` или `IPC_PRIVATE`
- ❑ `msgflg` – флаги, комбинируются с помощью `|` «битовое или»

Получение доступа / создание очереди сообщений SysV IPC

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>

int msgget(
    key_t key,
    int msgflg
);
```

❑ **msgflg** – флаги, комбинируются с помощью | «битовое или»

- IPC_CREAT - создать если не существует
- IPC_EXCL - вместе с предыдущим, создавать эксклюзивно, ошибка если существует
- Права доступа
- 0400 – только чтение для владельца
- 0200 – только запись для владельца
- 0100 – только исполнение для владельца
- 0040 – только чтение для группы
- ...

Отправка сообщения SysV IPC

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>

int msgsnd(
    int msqid,
    struct msgbuf* msgp,
    size_t msgsz,
    int msgflg
);
```

Помещает сообщение в очередь

❑ Возвращаемое значение

0 – успех

-1 – ошибка, код ошибки в **errno**

❑ **msqid** – идентификатор SysV IPC очереди

❑ **msgp** – указатель на сообщение, данные по шаблону `struct msgbuf`

❑ **msgsz** – размер полезной части сообщения, сообщение МИНУС атрибут типа, может =0

❑ **msgflg** – флаг

0 – по-умолчанию

IPC_NOWAIT – не ждать освобождения очереди

Шаблон сообщения SysV IPC

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>

struct msgbuf {
    long mtype;
    char mtext[1];
};
```

Шаблон структуры сообщения

- ❑ **mtype** – тип сообщения
обязательный атрибут, **> 0**
- ❑ **mtext** – заготовка полезной части сообщения
в Linux размер полезной части ≤ 4080

Примеры сообщений SysV IPC

```
struct mymsg1 {  
    long mtype;  
    char text[512];  
};
```

```
struct mymsg2 {  
    long mtype;  
    struct {  
        int n;    float x;  
    } info;  
};
```

- ❑ **mtype** – обязательное поле **>0**
- ❑ все данные после mtype – полезная часть
- ❑ размер полезной части
sizeof(struct mymsg) - sizeof(long)

Пример отправки сообщения

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <string.h>

struct mymsg { long mtype;  char text[512];  };

int main(int argc, char* argv[], char* envp[])
{
    key_t k = ftok("mylabel", 0);  if( k < 0 ){ ... }
    int msqid = msgget(k, IPC_CREAT|0664);  if( msqid < 0 ){ ... }

    struct mymsg msg;  msg.mtype = 1;  strcpy(msg.text, "hello");
    if( msgsnd(msqid, &msg, sizeof(msg)-sizeof(msg.mtype), 0) < 0 ){ ... }
    // ...
    return 0;
}
```


Получение сообщения SysV IPC

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>

int msgrcv(
    int msqid,
    struct msgbuf* msgp,
    size_t msgsz,
    long msgtype,
    int msgflg
);
```

Выбирает сообщение из очереди (копирует и удаляет)

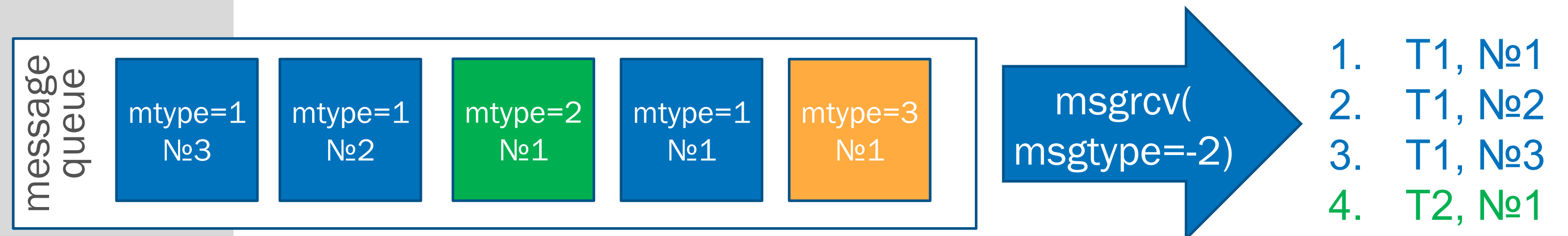
- ☐ Возвращаемое значение
 - >0 – размер полезной части полученного сообщения;
 - 1 – ошибка, код в **errno**
- ☐ **msqid** – идентификатор SysV IPC очереди
- ☐ **msgp** – буфер для размещения вычитываемого сообщения
- ☐ **msgsz** – максимальный размер полезной части сообщения, которое поместится в буфер
- ☐ **msgflg** – флаг
 - 0 – по-умолчанию
 - IPC_NOWAIT – не ждать появления сообщения
 - MSG_NOERROR – выбирать сообщение, даже если в буфере нет места

Получение сообщения SysV IPC

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>

int msgrcv(
    int msqid,
    struct msgbuf* msgp,
    size_t msgsz,
    long msgtype,
    int msgflg
);
```

- **msgtype** – способ выборки сообщения
 - 0 – в порядке FIFO независимо от типа
 - +n – в порядке FIFO только сообщения с типом n
 - n – в порядке FIFO для сообщений с типом $\leq n$: сначала друг за другом все сообщения с минимальным типом, потом все с большим типом, вплоть до n
- Первым выбирается сообщение с минимальным типом, не превышающим некоторого заданного значения, пришедшее раньше других сообщений с тем же типом.



Пример получения сообщения

```
#include <sys/types.h>    #include <sys/ipc.h>    #include <sys/msg.h>

struct mymsgbuf { long mtype;  char bytes[512];  } buf;
struct mymsg1 { long mtype;  struct { int n; float x; } info;  };

int main(int argc, char* argv[], char* envp[])
{
    key_t k = ftok("mylabel", 0);  if( k < 0 ){ ... }
    int msqid = msgget(k, IPC_CREAT|0664);  if( msqid < 0 ){ ... }

    if( msgrcv(msqid, &buf, sizeof(buf.bytes), 0/*type*/, 0) < 0 ){ ... }
    if( buf.mtype == 1 ) {
        struct mymsg1* msg = (struct mymsg1*)&buf;
        // msg->info.n
    }
    return 0;
}
```

Удаление очереди сообщений

SysV IPC

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>

int msgctl(
    int msqid,
    int cmd,
    struct msqid_ds* buf
);
```

Получает информацию об очереди сообщений, изменяет атрибуты, удаляет

❑ Возвращаемое значение

0 – успех

-1 – ошибка, код ошибки в **errno**

❑ **msqid** – идентификатор SysV IPC очереди

❑ **cmd** – код команды

IPC_RMID – для удаления

❑ **buf** – выходные/выходные данные для команд
NULL в случае команды удаления

Консольная команда для удаления очереди сообщений SysV IPC

```
$ ipcrm -q MSG_QUEUE_ID  
$ ipcrm -Q MSG_QUEUE_KEY
```

Удаление очереди сообщений SysV IPC

- ❑ **-q** id – удалить очередь по идентификатору
- ❑ **-Q** key – удалить очередь по ключу

POSIX Message Queues

- ❑ Более современный способ работы с очередями сообщений. Альтернатива System V Message Queues. В рамках нашего курса НЕ ИСПОЛЬЗУЕМ.
- ❑ Пространство имён очередей = имена из файловой системы
- ❑ `mq_open("/mymq", ...)`: создать или открыть очередь по имени (спец. файл обычно в `/dev/mqueue`).
- ❑ `mq_send(...)`: отправить сообщение.
- ❑ `mq_receive(...)`: прочитать сообщение.
- ❑ `mq_close(...)`: закрыть очередь.
- ❑ `mq_unlink()`: удалить очередь.

Мультиплексирование информации

- ❑ Возможность одновременного обмена информацией с несколькими партнёрами через одно средство связи
- ❑ Очередь сообщений SysV IPC возможно использовать для мультиплексирования
 - каждому процессу присписать свой собственный тип сообщений: `mtype == pid`
 - один процесс может получать сообщения от множества других процессов через одну очередь сообщений
 - процесс может отправлять ответы через ту же очередь сообщений
- ❑ Мультиплексирование часто применяется в модели взаимодействия **клиент - сервер**

Модель клиент – сервер

- ❑ Взаимодействующие процессы **неравноправны**
- ❑ Сервер работает постоянно, а клиенты могут работать эпизодически.
- ❑ Сервер ждет запроса от клиентов, инициатором же взаимодействия является клиент.
- ❑ Клиент обращается к одному серверу за раз, в то время как к серверу могут одновременно поступать запросы от нескольких клиентов.
- ❑ Клиент должен знать, как обратиться к серверу (например, какого типа сообщения он воспринимает) перед началом организации запроса к серверу, в то время как сервер может получить недостающую информацию о клиенте из пришедшего запроса.

Литература

1. В.Е. Карпов, К.А. Коньков - Основы операционных систем // М.: Национальный открытый университет «ИНТУИТ», 2016
2. В.Е. Карпов, К.А. Коньков - Основы операционных систем. Практикум // М.: Национальный открытый университет «ИНТУИТ», 2016
<https://intuit.ru/studies/courses/2249/52/info>
3. Google 😊

